

ChipChat Tutorial Assignment Report

Course: LLM4ChipDesign

Instructors: Prof. Ramesh Karri

Overview

This report documents the completion of the ChipChat Tutorial Assignment for the course LLM4ChipDesign. The assignment involved solving tutorial examples using a large language model (LLM), iteratively refining the generated Verilog designs to meet functional and syntactic requirements, and implementing an independent extension design. All final designs were verified using the provided or self-checking testbenches and compiled using iverilog.

Model and Configuration

The model used for this assignment was meta/llama-3.1-8b-instruct, deployed using NVIDIA NIM through the NVIDIA API available at build.nvidia.com. The model was executed with a maximum token limit of 1024. The temperature parameter was left at its default value, and streaming was disabled. The LLM was used to generate initial design drafts, which were then manually corrected to ensure synthesizability, correctness, and compatibility with the testbenches.

Part I – Tutorial Examples

Example A: binary_to_bcd

The first tutorial example involved designing a combinational binary-to-BCD converter. The design converts a 5-bit binary input in the range 0 to 31 into an 8-bit BCD output, where the upper 4 bits represent the tens digit and the lower 4 bits represent the ones digit. The module interface consists of a 5-bit binary input and an 8-bit BCD output.

The final implementation uses the double-dabble (shift-and-add-3) algorithm. Over five iterations, the BCD digits are adjusted by adding three when a digit is greater than or equal to five, then shifted left while shifting in the next binary input bit. The entire design is combinational and implemented using an always @* block. The initial LLM-generated output contained incorrect syntax and port mismatches, so it was replaced with a correct, synthesizable implementation. The final design compiled successfully with iverilog using the SystemVerilog 2012 standard and passed all provided test cases.

Example B: sequence_detector

The second tutorial example required designing a sequence detector that identifies a specific sequence of eight 3-bit input values: 001, 101, 110, 000, 110, 110, 011, and 101. The module interface includes a clock, an active-low reset, a 3-bit data input, and a sequence_found output.

The final design uses an eight-state finite state machine, with each state corresponding to one step in the target sequence. Separate states were introduced to correctly handle repeated values in the sequence. State transitions occur on the rising edge of the clock, with an active-low reset. To ensure compatibility with iverilog, state encoding was implemented using localparam definitions and a reg variable rather than SystemVerilog enum types. The sequence_found signal is generated combinatorially so that it is asserted in the correct cycle relative to the testbench sampling. After iterative debugging, the final design compiled successfully and passed all test cases.

Part II – Iterative Debugging Summary

During the development of the sequence detector, multiple iterations were required to resolve issues in the LLM-generated output. The first iteration had incorrect port names and attempted to detect the entire sequence as a single 8-bit value rather than an ordered sequence of 3-bit inputs. This was corrected by implementing a proper multi-state FSM. The second iteration introduced compilation errors in iverilog due to enum casting, which were resolved by replacing enums with localparam-based state encoding. In the third iteration, the sequence_found signal was moved from a sequential block to a combinational assignment to align with testbench timing expectations. After this change, all test cases passed.

Part III – Extension Design: Up/Down Counter

For the extension task, an 8-bit up/down counter with enable and wrap-around behavior was implemented. The module includes a clock, an active-low reset, an enable signal, an up_down control signal, and an 8-bit count output.

When enable is asserted and up_down is high, the counter increments. When enable is asserted and up_down is low, the counter decrements. When enable is deasserted, the counter holds its current value. The counter wraps from 255 to 0 when counting up and from 0 to 255 when counting down. The reset is active-low and synchronous. The design is implemented using a single always block with non-blocking assignments and explicit conditional logic to handle wrap-around behavior. The design is fully synthesizable.

A self-checking testbench was written to verify reset behavior, counting up, counting down, hold behavior, and wrap-around in both directions. Small delays were included in the testbench to ensure that non-blocking assignments were applied before comparisons were made. The counter design compiled successfully and passed all test cases.

Prompt Design Decisions

To improve the correctness of the generated designs and reduce ambiguity, several prompt design decisions were made. Exact port names were specified to match the testbenches. Wrap-around behavior at boundary values was described explicitly. The behavior when enabled is low was clearly stated. Active-low reset naming conventions consistent with the tutorial were used. The prompt explicitly requested synthesizable designs to avoid non-hardware constructs such as delays or initial blocks.

Compilation and Verification

All designs were compiled and verified using iverilog. The `binary_to_bcd` design, the `sequence_detector` design, and the up/down counter extension design each compiled without errors and passed their respective testbenches.

Conclusion

All tutorial examples and the extension design were completed, debugged, and verified. The assignment demonstrated that while LLMs can generate useful initial design drafts, careful prompting, iterative refinement, and manual validation are necessary to achieve correct and synthesizable hardware designs. All assignment requirements were met, and all tests passed.